Practically Groovy: Ant scripting with Groovy

Combine Ant and Groovy for more expressive and controllable builds

Skill Level: Introductory

Andrew Glover (andrew@thirstyhead.com)
Co-Founder
ThirstyHead.com

Scott Davis (scott@thirstyhead.com)
Founder
ThirstyHead.com

14 Dec 2004

Both Ant and Maven rule the world of build processing, but XML is occasionally a less-than-expressive configuration format. In this second installment in his new series on the practical applications of Groovy, Andrew Glover introduces Groovy's builder utility, which makes it especially easy to combine Groovy with Ant and Maven for more expressive and controllable builds.

The ubiquity and utility of Ant as a build tool for Java projects is virtually unsurpassed. Even Maven, the new, upstart utility in the build arena, owes much of its power to lessons learned from Ant. Where these two tools fall short, however, is extensibility. Even as the portability of XML has played a major role in propelling both Ant and Maven to the forefront of development, its role as build configuration format has somewhat limited the expressiveness of the resulting build process.

For example, while conditional logic is possible in both Ant and Maven, it is somewhat cumbersome using XML. Additionally, while it is possible to extend Ant's build process by defining custom tasks, doing so typically confines application behavior to a limited sandbox. So, in this article I'll show you how to combine Groovy with Ant *inside* Maven for greater expressiveness and a finer degree of behavioral control in the build process.

As you'll soon see for yourself, Groovy brings compelling enhancements to both Ant and Maven; and the beauty of it is that Groovy often picks up right where XML leaves off! In fact, it seems that Groovy's creators must have felt the pain of implementing Ant and Maven build processes in XML because they've introduced `AntBuilder`, a powerful new tool that supports the utilization of Ant within Groovy scripts.

In this installment of *Practically Groovy*, I'll show you how easy it is to enhance the build process by using Groovy rather than XML as your build configuration format. Closures are an important feature of Groovy and central to the language's expressiveness, so I'll start with a quick review of closures before moving on.

Quick review of closures

Much like some of its well known predecessors, Groovy supports the notion of *nameless functions* or *closures*. You may be familiar with closures if you've done any coding in Python or Jython, where a closure is introduced using the `lambda` keyword. In Ruby, you would have to really work to write a script *without* utilizing blocks and/or closures. Even the Java language supports a limited form of nameless functions, with its *anonymous inner classes*.

In Groovy, closures are first-class nameless functions that can encapsulate behavior. Ruby creator Yukihiro Matsumoto put his finger on the utility of closures when he noted that these powerful first class objects can be passed "to another function, and then that function can invoke the passed-in [closure]" (see Resources for the complete interview). Of course, playing with Groovy yourself is the best way to see what a powerful asset closures are to the niftiness of this exciting language.

About this series

The key to incorporating any tool into your development practice is knowing when to use it and when to leave it in the box. Scripting languages can be an extremely powerful addition to your toolkit, but only when applied properly to appropriate scenarios. To that end, *Practically Groovy* is a series of articles dedicated to exploring the practical uses of Groovy, and teaching you when and how to apply them successfully.

Closures in action

In Listing 1 I've reworked some of the code from the first article in this series; but this time with the emphasis on closures. If you've read that article (see Resources), you should recall the Java-based package filtering objects I used to demonstrate unit testing in Groovy. I'll start with the same example this time, but enhance it considerably with closures. Here, again, is the Java-based `Filter` interface.

Listing 1. Remember this one? A simple Java Filter interface

```
public interface Filter {
  void setFilter(String fltr);
  boolean applyFilter(String value);
}
```

After defining my `Filter` type last time, I proceeded to define two implementations, namely a `RegexPackageFilter` and a `SimplePackageFilter`, which applied regular expressions and simple `String` operations, respectively.

So far so good, for code written without closures. In Listing 2, you can begin to see how the code would have been different (and better!) with just a few syntactical changes. I'll start by defining the generic `Filter` type shown below, but this time in Groovy. Notice the `strategy` attribute that is now associated with the `Filter` class. This attribute is an instance of a closure that will be called when the `applyFilter` method is executed.

Listing 2. A Groovier Filter -- with closures

```
class Filter{
   def strategy
   boolean applyFilter(String str){
      return strategy.call(str)
   }
}
```

Adding closures means that I don't have to define an interface type as I did in the original `Filter` interface, or rely on specific implementations for desired behaviors. Instead, I can define a generic `Filter` type and give it a closure to be applied during the execution of the `applyFilter` method.

My next step is to define the two closures. The first one emulates the `SimplePackageFilter` (from the previous article) by applying simple `String` operations on a given parameter. When a new `Filter` type is created, the corresponding closure named `simplefilter` will be passed into the constructor. The second closure (which comes in after a few `asserts` for code validation) applies regular expressions to a given `String`. Again, I'll create a new `Filter` type, pass in the regular expression closure named `rfilter` and execute some `asserts` to ensure that everything is peachy. All of this is shown in Listing 3:

Listing 3. Simple magic with Groovy closures

```
simplefilter = { str ->
   if(str.indexOf("java.") >= 0){
     return true
   }else{
     return false
```

```
   }
}

fltr = new Filter(strategy:simplefilter)
assert !fltr.apply("test")
assert fltr.apply("java.lang.String")

rfilter = { istr ->
   if(istr =~ "com.vanward.*"){
     return true
   }else{
     return false
   }
}

rfltr = new Filter(strategy:rfilter)
assert !rfltr.apply("java.lang.String")
assert rfltr.apply("com.vanward.sedona.package")
```

Pretty impressive, huh? With closures, I was able to defer the definition of my desired behavior until runtime -- so there was no need to define a new `Filter` type and compile it, as I would have with my previous design. While I could do something similar with Java code using anonymous inner classes, it's simply easier and less esoteric in Groovy.

Closures are indeed powerful stuff. They also present a different way to handle behavior -- something Groovy (and its distant cousin Ruby) rely on heavily. Closures, consequently are key to my next topic, which is building with builders.

Building with builders

Central to the niftiness of Ant within Groovy is the notion of *builders*. Essentially, builders allow you to easily represent nested tree-like data structures, such as XML documents, in Groovy. And that, ladies and gentlemen, is the hook: With a builder, specifically an `AntBuilder`, you can effortlessly construct Ant XML build files *and execute the resulting behavior* without having to deal with the XML itself. And that's not the only advantage of working with Ant in Groovy. Unlike XML, Groovy is a highly expressive development environment wherein you can easily code looping constructs, conditionals, and even explore the power of reuse, rather than the cut-and-paste drill you may have previously associated with creating new build.xml files. And you still get to do it all within the Java platform!

The beauty of builders, and especially Groovy's `AntBuilder`, is that their syntax follows a logical progression closely mirroring that of the XML document they represent. Methods attached to an instance of `AntBuilder` match the corresponding Ant task; likewise, those methods can take parameters (in the form of a `map`) that correspond to the task's attributes. What's more, nested tags such as `includes` and `filesets` are then defined as closures.

Building blocks: Example 1

I'll introduce you to builders with a super easy example: an Ant task called `echo`. In Listing 4, I've created a normal, everyday XML version of Ant's `echo` tag (no surprises here):

Listing 4. Ant's Echo task

```
<echo message="This was set via the message attribute"/>
<echo>Hello World!</echo>
```

Things get more interesting in Listing 5, where I've taken that same Ant tag and redefined it in Groovy, with the `AntBuilder` type. Note that I can use `echo`'s attribute, `message`, or I can simply pass in a desired `String`.

Listing 5. Ant's Echo task in Groovy

```
ant = new AntBuilder()
ant.echo(message:"mapping it via attribute!")
ant.echo("Hello World!")
```

What's especially impressive about builders is how they let me mix in normal Groovy features with my builder syntax, thus creating a rich set of behaviors. In Listing 6 you should begin to see how endless are the possibilities:

Listing 6. Flow control with Groovy and Ant

```
ant = new AntBuilder()
ant.mkdir(dir:"/dev/projects/ighr/binaries/")

try{

    ant.javac(srcdir:"/dev/projects/ighr/src",
       destdir:"/dev/projects/ighr/binaries/" )

}catch(Throwable thr){
    ant.mail(mailhost:"mail.anywhere.com", subject:"build failure"){
       from(address:"buildmaster@anywhere.com", name:"buildmaster")
       to(address:"dev-team@anywhere.com", name:"Development Team")
       message("Unable to compile ighr's source.")
    }
}
```

In this example, I've attempted to capture an error condition when compiling source code. Notice how the `mail` object defined in the `catch` block takes a closure that defines the `from`, `to`, and `message` attributes.

Whew! That's a whole lot of Groovy. Of course, knowing when to apply such clever features is the crux of the matter, and all of us struggle with it from time to time. Fortunately, practice often does make perfect, and once you've begun to use Groovy (or any scripting language, for that matter) you'll likely find many opportunities to use

it in real life; and from that you'll learn where it really fits. In the next section I'll look at a typical, real-world example where I put some of the features introduced here to use.

Applied Grooviness

For the sake of this example, let's pretend for a moment that I need to create a checksum report for my code on a regular basis. Once I've implemented it, I can also use the report to verify file integrity if I ever need to. Here's a high-level technical use case for the checksum report:

1. Compile all source code.
2. Run an md5 algorithm against the binary class files.
3. Create a simple report that lists each class file and its corresponding checksum.

Completely scrapping Ant or Maven and using Groovy for the entire build process would be a bit extreme in this case. Indeed, as I explained earlier, Groovy is a great *enhancement* to these tools, *not* a replacement. As such, it makes sense to tackle the last two items with the expressiveness of Groovy and trust the first step to Ant or Maven.

In fact, let's just assume that I've used Maven for the first step, because, quite frankly, it's my preferred build platform. Compiling source files is easy to do in Maven with the `java:compile` and `test:compile` goals; thus, I can leave them untouched and make my new goal reference the preceding goals as prerequisites. And that's it -- with the source files compiled I'm ready to move on to running the checksum utility; but first I need to do some simple setup.

Setting up Md5ReportBuilder

In order to run this nifty checksum tool via Ant, I need two pieces of information: which directories contain the files to run a checksum against and what directory the report should be written to. For the first argument to the utility I will expect a comma-separated list of directories. For the last argument I'll expect the desired report directory.

I've decided to call the utility class `Md5ReportBuilder`. Its `main` method is defined in Listing 7:

Listing 7. Main method in Md5ReportBuilder

```
class Md5ReportBuilder{
  static void main(String[] args) {
    assert args[0] && args[1] != null

    def dirs = args[0].split(",")
    def todir = args[1]

    def report = new Md5ReportBuilder()
    report.runCheckSum(dirs)
    report.buildReport(dirs, todir)
  }
}
```

My first step above is to check that my two arguments have been passed into the utility. I then create an array of directories to run Ant's `checksum` task against by splitting the first argument with a comma. Finally, I create a new instance of the `Md5ReportBuilder` class, calling two methods to handle the needed functionality.

Adding the checksum

Ant includes a `checksum` task that can be invoked very easily, by passing in a desired `fileset` containing a collection of target files. In this case, the target files are the directories containing compiled source files and corresponding unit test files. I can get at these files by iterating with a `for` loop, which in this case iterates over the collection of directories. For every directory, the `checksum` task is called; moreover, the `checksum` task is only run on .class files, as shown in Listing 8:

Listing 8. The runCheckSum method

```
/**
 * runs checksum task for each dir in collection passed in
 */
void runCheckSum(String[] dirs){
  def ant = new AntBuilder()
  dirs.each{idir->
    ant.checksum(fileext:".md5.txt" ){
      fileset(dir:idir) {
        include(name:"**/*.class")
      }
    }
  }
}
```

Building the report

From this point building the report becomes an exercise in looping. Each newly generated checksum file is read and its corresponding information is fed to a `PrintWriter` that writes the XML to a file -- -- albeit in a most ugly fashion -- as shown in Listing 9:

Listing 9. Building a report

```
void buildReport(String[] dirs, String todir){
  def ant = new AntBuilder()
  dirs.each{bsedir ->
    def scanner = ant.fileScanner {
      fileset(dir:bsedir) {
        include(name:"**/*.class.md5.txt")
      }
    }

    def rdir = todir + File.separator + bsedir + File.separator + "xml" + File.separator
    def file = new File(rdir)

    if(!file.exists()){
      ant.mkdir(dir:rdir)
    }

    def nfile = new File(rdir + File.separator + "checksum.xml")

    nfile.withPrintWriter{ pwriter ->
       pwriter.println("<md5report>")
       scanner.each{ f ->
         f.eachLine{ line ->
           pwriter.println("<md5 class='" + f.path + "' value='" + line + "'/>")
         }
       }
       pwriter.println("</md5report>")
    }
  }
}
```

So what's going on in this report? First, I've used a `FileScanner` to find every checksum file created from the `checksum` method from Listing 8. Next, I've created a new directory and a new file inside that directory. (Isn't it nice how I can check if the directory exists via a simple `if` statement?) I've then opened the corresponding file and -- using a nifty closure -- read each corresponding `File` from the `scanner` collection. The example concludes with the report contents being written into an XML element.

Goals are your target

Readers not familiar with Maven may wonder what all this talk of `goals` is about. Think of `goals` in Maven as `targets` in Ant. A *goal* is simply a way to group behavior. Maven `goals` are given names that can be called via the `maven` command. Any tasks found in a given `goal` are executed upon being called.

I'll bet you noticed right away the powerful effect of the `withPrintWriter` method on those instances of `File`. I didn't need to worry about exceptions or closing the file because everything was handled for me. I just passed in my desired behavior via a closure and was good to go!

Running the utility

The next stop on this Groovy roadshow is to plug it into a build process, specifically my maven.xml file. Luckily, this is the easiest part of a relatively easy exercise, as shown in Listing 10:

Listing 10. Running Md5ReportBuilder in Maven

```
<goal name="gmd5:run" prereqs="java:compile,test:compile">

  <path id="groovy.classpath">
    <ant:pathelement path="${plugin.getDependencyClasspath()}"/>
    <ant:pathelement location="${plugin.dir}"/>
    <ant:pathelement location="${plugin.resources}"/>
  </path>

  <java classname="groovy.lang.GroovyShell" fork="yes">
    <classpath refid="groovy.classpath"/>
    <arg value="${plugin.dir}/src/groovy/com/vanward/md5builder/Md5ReportBuilder.groovy"/>
    <arg value="${maven.test.dest},${maven.build.dest}"/>
    <arg value="${maven.build.dir}/md5-report"/>
  </java>
</goal>
```

As explained earlier, I've stipulated that the checksum utility must run after a full compile; hence, my goal has two prerequisites: `java:compile` and `test:compile`. Classpaths are always important, so I've taken special care to create a proper classpath for Groovy to run. Lastly, the goal shown in Listing 10 invokes Groovy's shell, passing in the script to run and the corresponding two arguments it accepts -- firstly the (comma-delimited) directories to run the checksum against and secondly the directory to which the report should be written.

No loose ends!

Once I've coded the maven.xml file I'm almost done, but there is one last step: I need to update the `project.xml` file with the required dependencies. It should be no surprise that Maven needs to have Groovy's required dependencies to work. Those dependencies are asm, a byte code manipulation library, commons-cli (which handles command-line parsing) Ant, and the associated ant-launcher. The dependencies for this example are shown in Listing 11:

Listing 11. Groovy's required dependencies

```
<dependencies>
  <dependency>
    <groupId>groovy</groupId>
    <id>groovy</id>
    <version>1.0-beta-6</version>
  </dependency>

  <dependency>
    <groupId>asm</groupId>
    <id>asm</id>
```

```
    <version>1.4.1</version>
  </dependency>

  <dependency>
    <id>commons-cli</id>
    <version>1.0</version>
  </dependency>

  <dependency>
    <groupId>ant</groupId>
    <artifactId>ant</artifactId>
    <version>1.6.1</version>
  </dependency>

  <dependency>
    <groupId>ant</groupId>
    <artifactId>ant-launcher</artifactId>
    <version>1.6.1</version>
  </dependency>
</dependencies>
```

Lesson review

In this second installment of *Practically Groovy*, you've seen what happens when the expressiveness and agility of Groovy is coupled with the unbeatable utility of Ant and Maven. For either tool, Groovy offers a compelling build-format alternative to XML. It greatly enhances the build process by letting you control program flow with looping constructs and conditional logic.

Along with showing you the practical side of Groovy, this series is dedicated to exploring its most appropriate uses. One of the keys to using any technology is carefully considering the context in which it's going to be applied. In this case, I've shown you how Groovy can be used to *enhance* rather than *replace* an already very powerful utility: Ant.

Next month I'll introduce another Groovy feature that relies on closures. GroovySql is a super handy little utility that makes database queries, updates, inserts, and all the corresponding logic exceptionally easy to manage. See you then!

Downloads

Description	Name	Size	Download method
Sample code	j-pg12144.zip	7KB	HTTP

Information about download methods

About the authors

Andrew Glover
Andrew Glover is a developer, author, speaker, and entrepreneur. He is the founder of the easyb Behavior-Driven Development (BDD) framework and is the co-author of three books: Continuous Integration, Groovy in Action, and Java Testing Patterns. He teaches a wide variety of Groovy-, Grails-, and testing-related classes at ThirstyHead.com. You can keep up with Andy at thediscoblog.com where he routinely blogs about software development.

Scott Davis
Scott Davis is an internationally recognized author, speaker, and software developer. He is the founder of ThirstyHead.com, a Groovy and Grails training company. His books include Groovy Recipes: Greasing the Wheels of Java, GIS for Web Developers: Adding Where to Your Application, The Google Maps API, and JBoss At Work. He writes two ongoing article series for IBM developerWorks: Mastering Grails and Practically Groovy.